

Image CLipper — Relatório do Trabalho 01

Disciplina: SCC0251 — Processamento de Imagens

Profa.: Leo Sampaio Ferraz Ribeiro

Aluno: Cauê Paiva Lira — 14675416

Semestre: 1/2026

1. Introdução

O **Image CLipper** é um editor de imagens via linha de comando (CLI) que implementa transformações geométricas e de intensidade.

O programa recebe uma imagem de entrada, aplica a transformação desejada e salva o resultado em um arquivo de saída. Todas as operações são implementadas do zero usando apenas **numpy** e **imageio**, incluindo a interpolação bilinear utilizada nas transformações geométricas.

2. Instruções de Uso

2.1 Requisitos

- Python 3.10+
- Bibliotecas: **numpy**, **imageio**

2.2 Sintaxe Geral

```
python editor.py <comando> -i <entrada> -o <saida> [opcoes]
```

2.3 Comandos Disponíveis

Comando	Descrição	Opções
translate	Translação com wrap-around	--dx , --dy
rotate	Rotação em torno do centro	--angle , --strategy (obrigatório)
scale	Escala por fator	--factor
crop	Recorte de região	--x , --y , --w , --h
inverse	Inversão de intensidade	—
log	Transformação logarítmica	—
gamma	Correção gamma	--gamma
contrast	Modulação de contraste piecewise	--intervals , --targets
creative	Solarização	--threshold

2.4 Exemplos de Execução

Transformações de intensidade

```
python editor.py inverse -i foto.png -o invertida.png
python editor.py log -i foto.png -o log.png
python editor.py gamma -i foto.png -o gamma.png --gamma 2.2
python editor.py contrast -i foto.png -o contraste.png --intervals "0,128,255" --targets "0,200,255"
python editor.py creative -i foto.png -o solar.png --threshold 128
```

Transformações geométricas

```
python editor.py translate -i foto.png -o movida.png --dx 50 --dy 30
python editor.py scale -i foto.png -o grande.png --factor 2.0
```

```
python editor.py crop      -i foto.png -o recorte.png --x 10 --y 10 --w 200 --h 150
```

```
# Rotação com diferentes estratégias
```

```
python editor.py rotate -i foto.png -o rot.png --angle 30 --strategy autozoom
```

```
python editor.py rotate -i foto.png -o rot.png --angle 45 --strategy nearest
```

3. Transformações Geométricas

3.1 Translação

Desloca a imagem em dx pixels na horizontal e dy pixels na vertical. Utiliza **wrap-around** (`numpy.roll`) para evitar pixels vazios — os pixels que “saem” de um lado reaparecem no lado oposto.

Imagem original:



Figura 1: Original

Resultado ($dx=200$, $dy=150$):

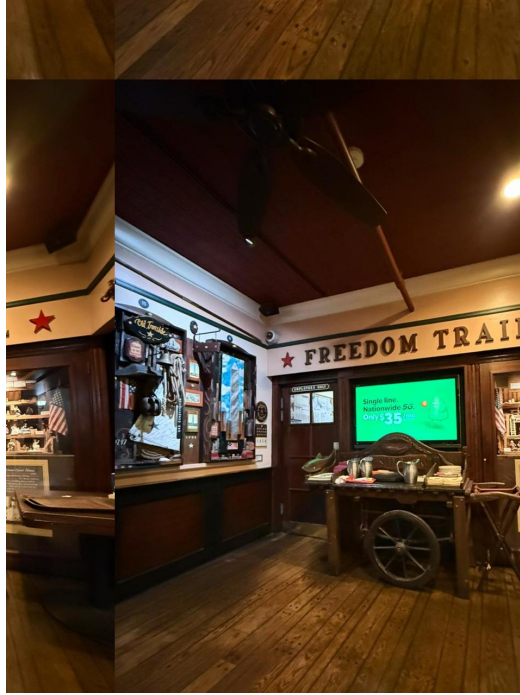


Figura 2: Translação

3.2 Rotação

Rotação em torno do centro da imagem por um ângulo em graus (sentido anti-horário). Implementada com **mapeamento inverso** e **interpolação bilinear** manual.

Para cada pixel (x', y') da imagem de saída, calcula-se a coordenada de origem:

$$\begin{aligned} x_{src} &= \cos(\theta)(x' - c_x) + \sin(\theta)(y' - c_y) + c_x \\ y_{src} &= -\sin(\theta)(x' - c_x) + \cos(\theta)(y' - c_y) + c_y \end{aligned}$$

onde (c_x, c_y) é o centro da imagem e θ é o ângulo de rotação.

Tratamento de Pixels Pretos A rotação pode introduzir pixels pretos nas bordas. Para evitar isso, o usuário deve escolher obrigatoriamente uma das duas estratégias via **--strategy**:

- **autozoom**: calcula um fator de zoom que elimina as bordas pretas, amplia a imagem antes de rotacionar, e recorta ao tamanho original.
- **nearest**: estende os pixels da borda para preencher as áreas vazias.

Imagem original (rotação 30° autozoom):



Figura 3: Original

Rotação 30° (autozoom):



Figura 4: Rotação autozoom

Imagem original (rotação 30° nearest):



Figura 5: Original

Rotação 30° (nearest):



Figura 6: Rotação nearest

Imagem original (rotação 45° nearest):



Figura 7: Original

Rotação 45° (nearest):



Figura 8: Rotação nearest 45°

3.3 Escala

Redimensiona a imagem por um fator multiplicativo usando **interpolação bilinear**. As coordenadas de saída são mapeadas linearmente para a imagem original com `numpy.linspace`, e os valores são interpolados com a função `_bilinear_interp`.

Imagem original (escala 0.1x):



Figura 9: Original

Escala 0.1x:



Figura 10: Escala 0.1x

3.4 Crop (Recorte)

Recorta uma região retangular da imagem definida por posição (x, y) e dimensões (w, h) , usando slicing numpy: `img[y:y+h, x:x+w]`.

Imagem original:

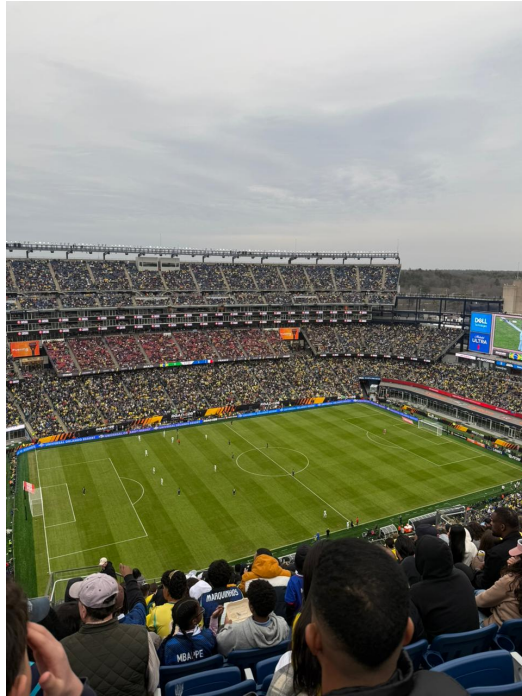


Figura 11: Original

Recorte (x=200, y=600, w=500, h=400):



Figura 12: Crop

4. Transformações de Intensidade

4.1 Inversa

Inverte os valores de intensidade: $f(x) = 255 - x$. Pixels claros tornam-se escuros e vice-versa.

Imagem original:



Figura 13: Original

Resultado:



Figura 14: Inversa

4.2 Log

Transformação logarítmica: $f(x) = c \cdot \ln(1 + x)$, onde $c = \frac{255}{\ln(256)}$. Expande valores baixos de intensidade e comprime valores altos, útil para realçar detalhes em regiões escuras.

Imagem original:



Figura 15: Original

Resultado:



Figura 16: Log

4.3 Gamma

Correção gamma: $f(x) = 255 \cdot \left(\frac{x}{255}\right)^\gamma$. Valores de $\gamma < 1$ clareiam a imagem; valores de $\gamma > 1$ escurecem.

Imagem original (gamma 3.5):



Figura 17: Original

Gamma = 3.5 (escurecimento intenso):



Figura 18: Gamma 3.5

Imagem original (gamma 2.2):



Figura 19: Original

Gamma = 2.2 (escurecimento moderado):



Figura 20: Gamma 2.2

4.4 Modulação de Contraste

Mapeamento piecewise linear definido pelo usuário. O usuário especifica pares de intervalos de entrada e valores de saída, e a função `numpy.interp` realiza a interpolação linear entre eles.

Exemplo: `--intervals "0,50,200,255" --targets "0,150,200,255"` expande o contraste na faixa escura e comprime na faixa clara.

Imagem original:



Figura 21: Original

Resultado:



Figura 22: Contraste

4.5 Solarização (Transformação Criativa)

Função criativa inspirada no efeito de solarização fotográfica. Pixels com intensidade abaixo de um limiar (threshold) permanecem inalterados; pixels acima do limiar são invertidos:

$$f(x) = \begin{cases} x & \text{se } x < T \\ 255 - x & \text{se } x \geq T \end{cases}$$

Isso cria um efeito surreal que mistura tons positivos e negativos na mesma imagem.

Imagem original:

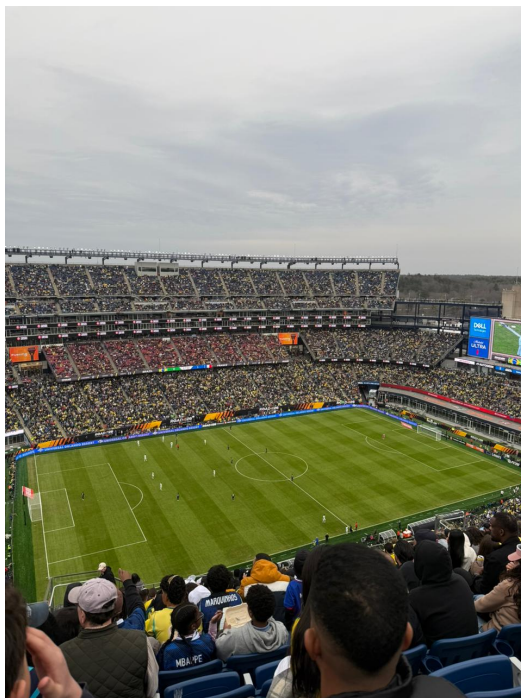


Figura 23: Original

Solarização (threshold=128):



Figura 24: Solarização

5. Detalhes de Implementação

5.1 Interpolação Bilinear

A função `_bilinear_interp` é o núcleo das transformações geométricas (rotação e escala). Para coordenadas fracionárias (x, y) , calcula a média ponderada dos 4 pixels vizinhos:

$$v = v_{00}(1 - f_y)(1 - f_x) + v_{01}(1 - f_y)f_x + v_{10}f_y(1 - f_x) + v_{11}f_yf_x$$

Suporta três modos de tratamento de borda: - **constant**: preenche com valor constante (padrão: 0) - **nearest**: replica o pixel mais próximo da borda - **wrap**: wrap-around (coordenadas cíclicas)

5.2 Organização do Código

O arquivo `editor.py` está organizado nas seguintes seções:

Seção	Funções
Helpers de I/O	<code>load_image</code> , <code>save_image</code>
Interpolação	<code>_bilinear_interp</code>
Transf. Geométricas	<code>translate</code> , <code>_rotate_core</code> , <code>rotate_image</code> , <code>scale_image</code> , <code>crop_image</code>
Detecção/Autozoom	<code>detect_black_pixels</code> , <code>compute_autozoom_factor</code>
Transf. de Intensidade	<code>inverse</code> , <code>log_transform</code> , <code>gamma_transform</code> , <code>contrast_transform</code> , <code>solarize</code>
CLI	<code>build_parser</code> , <code>main</code>

5.3 Bibliotecas Utilizadas

- **numpy** — operações matriciais, interpolação, manipulação de pixels
 - **imageio** — leitura e escrita de arquivos de imagem
 - **argparse** — parsing dos argumentos de linha de comando
-

6. Conclusão

O Image CLipper implementa todas as funcionalidades exigidas pelo trabalho: três transformações geométricas (translação, rotação e escala/crop) e cinco transformações de intensidade (inversa, log, gamma, modulação de contraste e solarização como transformação criativa). As transformações geométricas foram implementadas do zero, incluindo a interpolação bilinear, sem depender de funções prontas de bibliotecas como scipy. O software funciona tanto com imagens em escala de cinza quanto com imagens RGB coloridas.